

MODULE 3: SOFTWARE CONSTRUCTION 1.2
WEEK 3

Learning Outcomes:

- After completing this course, you are expected to demonstrate the following:
1. Clarify impact analysis in software testing and how to prevent a buffer overflow in the program.

A. Engage

Trivia

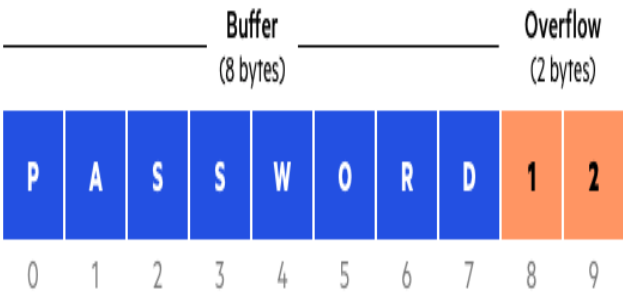


Figure 3.0

In information security and programming, a buffer overflow, or buffer overrun, is an anomaly where a program, while writing data to a buffer, overruns the buffer's boundary and overwrites adjacent memory locations.

B. Explore

Video Titles:

1. Impact Analysis Meeting in Software Development / Software Testing.
2. Ethical Hacking: Buffer Overflow Basics.
3. Day-9 #Read and #Write file using #BufferedReader and #BufferedWriter in #Java.

YouTube Links

1. https://www.youtube.com/watch?v=_1kyPtptvnU
2. <https://www.youtube.com/watch?v=SOoJcrR4Ijo>
3. <https://www.youtube.com/watch?v=iNSEX9IOqO4>

C. Explain

What is Impact Analysis?

Impact Analysis is defined as analyzing the impact of changes in the deployed product or application. It gives the information about the areas of the system that may be affected due to the change in the section or features of the application.

The impact is analyzed on Requirements, Design & Architecture, impact on Test and impact on schedule. With the incorporation of new features into application or product, it becomes imperative to check the influence of these new features or changes on the performance of the system. For this reason, Impact Analysis is done.

D. Elaborate

Why is Change Impact Analysis is done?

- ✓ It is done to understand the possible outcome of implementing the change. Including too much functionality into a product can reduce the overall performance of the product.
- ✓ To identify all the files, documents and models that might have to be modified if a team decides to implement the change in product.
- ✓ To estimate the effort needed behind implementing the change.
- ✓ To identify the task required to implement the change.
- ✓ It will list the dependencies on a specific element.

What is Impact Analysis Document?

Impact Analysis document can be used as a checklist. It is used to evaluate change request before working on them. The Impact Analysis document should give details like

- Brief description of an issue
- Explain or show an example of how the defect is causing failure and/or inefficiency.
- Include an estimate of complexity.
- Include an estimate of cost and time for a fix.
- Functionality to be tested.
- List down the new test cases created for the change.
- Reference document- Mention the reference document, technical specification, etc.

Example: Impact Analysis Document.

- 1) Change Request ID:
- 2) Title:
- 3) Description:
- 4) Date Prepare:

- 5) Prioritization estimate:
 - Relative Benefit
 - Relative Penalty
 - Relative cost
 - Relative risk
- 6) Estimated total effort: hours
- 7) Estimated lost effort: hours
- 8) Estimated schedule Impact: days
- 9) Quality Impacted
- 10) Other Requirement affected
- 11) Other tasks affected
- 12) Integration issues

How to present Impact Analysis influence level

Impact Analysis can be marked as per the color code to represent the criticality of the changes or impact of the changes over the system. The color code could be anything as shown below.

- a) Red- Strong
- b) Yellow- Moderate
- c) Green- Weak

Changes/Impact	Feature1	Feature2	Feature3	Feature4	Feature5	Feature6
Feature1	3	1	2			
Feature2						
Feature3						
Feature4				3	2	3
Feature5						
Feature6						

Figure 3.1

The above table explains the impact of the implemented changes.

- Features that are marked with red indicates the main features which are changed, those with yellow are the features that are less influenced by the change and those with the green are the least once.
- Features mentioned vertically resembles those features that are changed while horizontally those features are mentioned which performed changes can influence. For example, in above example the change in feature 1 influence the feature3.
- For a larger project where features and functionalities are more than above table may not be of any use. In such case, another approach is adopted, where developer

straightaway marks the level of influence due to the changes in main features. As shown below where the impact of the Main feature is marked for respective sub-features.

Main Features/Sub-Features	Sub-Feature1	Sub-Feature2	Sub-Feature3	Sub-Feature4	Sub-Feature5	S F
Main Feature1	3		2	1		

Figure 3.2

Sample Questions to be addressed for performing Impact Analysis

- ✓ What are the adverse side effects or risks of making the proposed change?
- ✓ Does any tool be acquired to implement and test the change?
- ✓ If the change is accepted how much effort will be lost that is already been invested?
- ✓ Does proposed changes adversely affect performance requirements?
- ✓ To verify the proposed change does another user input be required.
- ✓ Does the change increase the product cost?
- ✓ Does the proposed change is something the current staff has knowledge and skills?
- ✓ Does the proposed change place any unacceptable demand on any computer resource?

Best practices for change Impact Analysis

- ✓ Before initiating with Impact Analysis, make sure testing request contained no information about those part of the project that was influenced by changes.
- ✓ Continue communication between developer and tester is must, not to miss any change needed to implement in the final product.
- ✓ Identify if any user interface changes, deletions or additions are required.
- ✓ Estimate the number of acceptances, system or integration test cases that will be required.
- ✓ Identify any impact of the proposed change to another project plan, configuration management plan or quality assurance plan.

Buffer Overflow

What is Buffer Overflow?

Buffers are memory storage regions that temporarily hold data while it is being transferred from one location to another. A buffer overflow (or buffer overrun) occurs when the volume of data exceeds the storage capacity of the memory buffer. As a result, the program attempting to write the data to the buffer overwrites adjacent memory locations.

For example, a buffer for log-in credentials may be designed to expect username and password inputs of 8 bytes, so if a transaction involves an input of 10 bytes (that is, 2 bytes more than expected), the program may write the excess data past the buffer boundary.

Buffer overflows can affect all types of software. They typically result from malformed inputs or failure to allocate enough space for the buffer. If the transaction overwrites executable code, it can cause the program to behave unpredictably and generate incorrect results, memory access errors, or crashes.

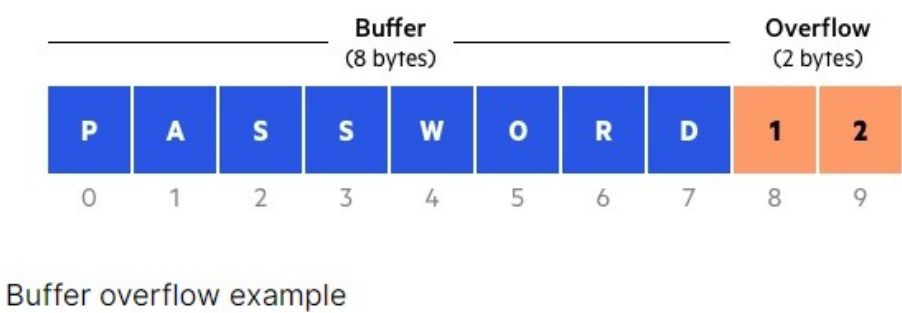


Figure 3.3

What is a Buffer Overflow Attack?

Attackers exploit buffer overflow issues by overwriting the memory of an application. This changes the execution path of the program, triggering a response that damages files or exposes private information. For example, an attacker may introduce extra code, sending new instructions to the application to gain access to IT systems.

If attackers know the memory layout of a program, they can intentionally feed input that the buffer cannot store, and overwrite areas that hold executable code, replacing it with their own code. For example, an attacker can overwrite a pointer (an object that points to another area in memory) and point it to an exploit payload, to gain control over the program.

Types of Buffer Overflow Attacks

- Stack-based buffer overflows are more common, and leverage stack memory that only exists during the execution time of a function.
- Heap-based attacks are harder to carry out and involve flooding the memory space allocated for a program beyond memory used for current runtime operations.

What Programming Languages are More Vulnerable?

C and C++ are two languages that are highly susceptible to buffer overflow attacks, as they do not have built-in safeguards against overwriting or accessing data in their memory. Mac OSX, Windows, and Linux all use code written in C and C++. Languages such as PERL,

Java, JavaScript, and C# use built-in safety mechanisms that minimize the likelihood of buffer overflow.

How to Prevent Buffer Overflows?

Developers can protect against buffer overflow vulnerabilities via security measures in their code, or by using languages that offer built-in protection. In addition, modern operating systems have runtime protection. Three common protections are:

- 1) **Address space randomization (ASLR)**—randomly moves around the address space locations of data regions. Typically, buffer overflow attacks need to know the locality of executable code, and randomizing address spaces makes this virtually impossible.
- 2) **Data execution prevention**—flags certain areas of memory as non-executable or executable, which stops an attack from running code in a non-executable region.
- 3) **Structured exception handler overwrites protection (SEHOP)**—helps stop malicious code from attacking **Structured Exception Handling (SEH)**, a built-in system for managing hardware and software exceptions. It thus prevents an attacker from being able to make use of the SEH overwrite exploitation technique. At a functional level, an SEH overwrite is achieved using a stack-based buffer overflow to overwrite an exception registration record, stored on a thread's stack.

Security measures in code and operating system protection are not enough. When an organization discovers a buffer overflow vulnerability, it must react quickly to patch the affected software and make sure that users of the software can access the patch.

How Imperva Helps Mitigate Buffer Overflow Attacks?

The Imperva security solution is deployed as a gateway to your application and provide out-of-the-box protection for buffer overflow attacks. It does so by blocking illegal requests that may trigger a buffer overflow state, preventing them from reaching your applications.

In addition to protecting against buffer overflow attacks, Imperva provides multi-layered protection to make sure websites and applications are available, easily accessible, and safe. The Imperva application security solution includes:

1. **DDoS Protection**—maintain uptime in all situations. Prevent any type of DDoS attack, of any size, from preventing access to your website and network infrastructure.
2. **Web Application Firewall**—permit legitimate traffic and prevent bad traffic. Safeguard your applications on-premises and at the edge with an enterprise-class cloud WAF.
3. **Bot Management**— get full visibility and control over human, good bot, and bad bot traffic to your website and API.
4. **Account Takeover Protection**—uses an intent-based detection process to identify and defends against attempts to take over users' accounts for malicious purposes.
5. **API Security**—protects APIs by ensuring only desired traffic can access your API endpoint, as well as detecting and blocking exploits of vulnerabilities.

- 6. **RASP**—keep your applications safe from within against known and zero-day attacks. Fast and accurate protection with no signature or learning mode.
- 7. **Attack Analytics**—mitigate and respond to real security threats efficiently and accurately with actionable intelligence across all your layers of defense.

E. Evaluate

ASSESSMENT:

Instructions: Write your answer on the Answer Sheet (AS) provided in this module. (60 pts)

1. Enumeration

- a. Give the five reasons why change impact analysis is done? (3 pts each)
- b. Give the five best practices for change impact analysis (3 pts each)
- c. Give the three ways to prevent buffer overflows with definition (5 pts each)
- d. Give at least five buffer overflow attacks with definition (3 pts each)

References:

- 1. *Impact Analysis in Software Testing*
<https://www.guru99.com/impact-analysis-software-testing.html>
- 2. *Buffer Overflow Attack*
<https://www.imperva.com/learn/application-security/buffer-overflow/>

Facilitated By:

Name	:	
MS Teams Account (email)	:	
Smart Phone Number	:	